DOCUMENTAÇÃO DO PROJETO

Hype Cards

Sistema web de controle de estoque e vendas para loja de Pokémon TCG



Murillo Daigder Pedrão Dias
Engenharia de Softeware

Londrina · 2026

Uma loja, três tipos de mercadoria

Cartas avulsas

Coleção, número, raridade, condição e idioma. Uma só carta pode valer milhares de reais.

Produtos selados

Boosters, Elite Trainer Boxes, displays e latas, sempre ligados a uma coleção.

Acessórios

Sleeves, deck boxes, pastas e playmats, por categoria e marca.

Sem sistema: quanto tenho em estoque? Quanto vendi e lucrei? Quem fez cada movimentação? Como desfaço uma venda errada?

Objetivos do sistema

01

Centralizar o cadastro de todos os itens, com foto

02

Vender com carrinho, desconto e recibo para o cliente

03

Estornar vendas sem apagar o histórico

04

Registrar cada entrada e saída com motivo, valor e usuário

05

Relatórios com faturamento, lucro e ticket médio

06

Alertar estoque baixo, proteger o acesso e fazer backup

Stack e arquitetura

Navegador

HTML5, CSS3 e JavaScript em 14 módulos · gráficos SVG sem bibliotecas

API Express 5 · Node.js 22

Regras de negócio e rotas REST · uma camada de acesso a dados

PostgreSQL 18

Produção, com transações

SQLite

Arquivo único, alternativo

Testes no GitHub Actions · banco escolhido por DATABASE_URL

2

dependências externas: express e pg

Hash scrypt, backup ZIP com zlib, CSV e gráficos feitos com módulos nativos do Node.

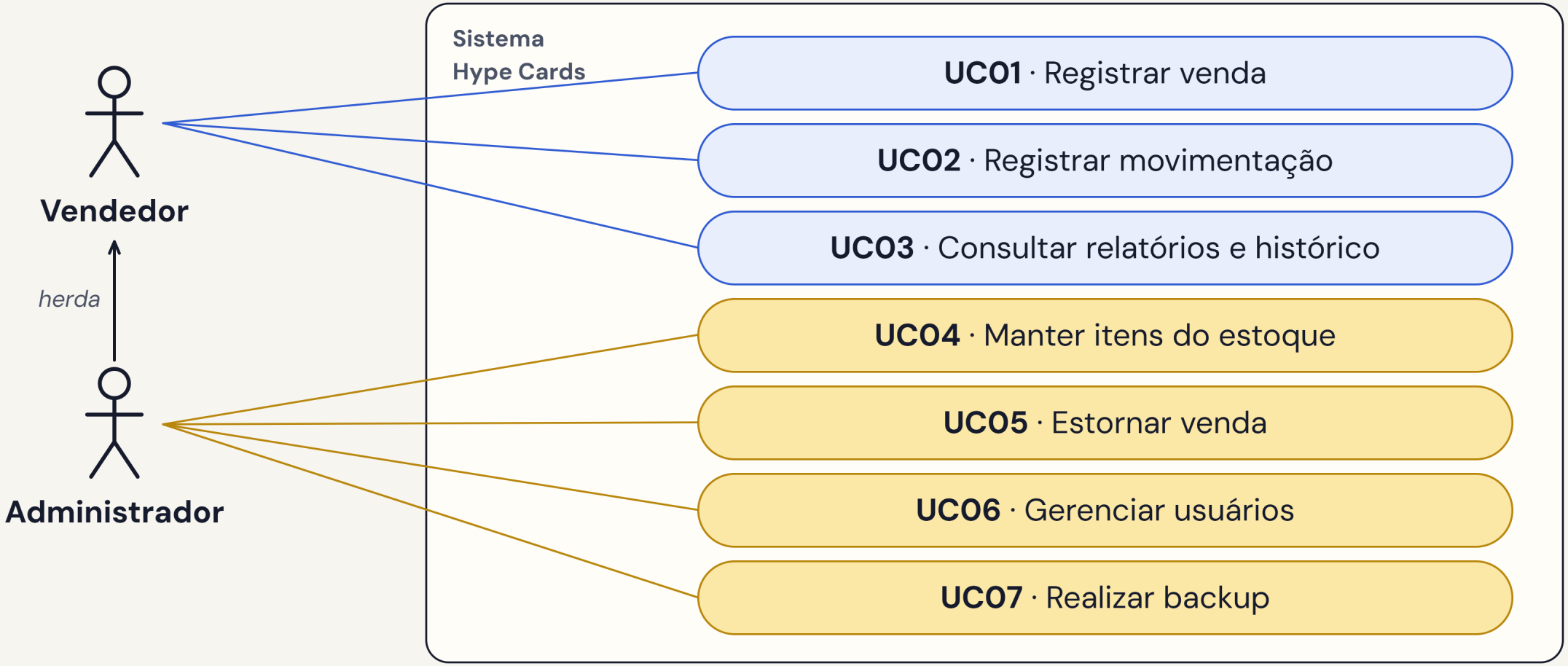
Requisitos funcionais · 11 de 24

Código	Requisito	Ator
RF01	Autenticar com login e senha e encerrar a sessão (logout)	Todos
RF03	Gerenciar usuários com perfil administrador ou vendedor	Administrador
RF04–07	Manter coleções, cartas avulsas, produtos selados e acessórios	Administrador
RF09–10	Registrar entrada e saída de estoque com motivo, sem saldo negativo	Todos
RF12	Consultar o histórico com filtros e link para a venda de origem	Todos
RF13	Painel do mês: vendas, lucro, estoque e itens abaixo do mínimo	Todos
RF17	Gerar backup em ZIP com o banco de dados e as imagens	Administrador
RF19	Registrar venda com carrinho, desconto rateado e forma de pagamento	Todos
RF20	Recusar a venda inteira quando faltar saldo de algum item	Sistema
RF22	Estornar venda com motivo, devolvendo os itens ao estoque	Administrador
RF24	Relatórios por período: faturamento, lucro, ticket médio e ranking	Todos

Requisitos não funcionais · 9 de 13

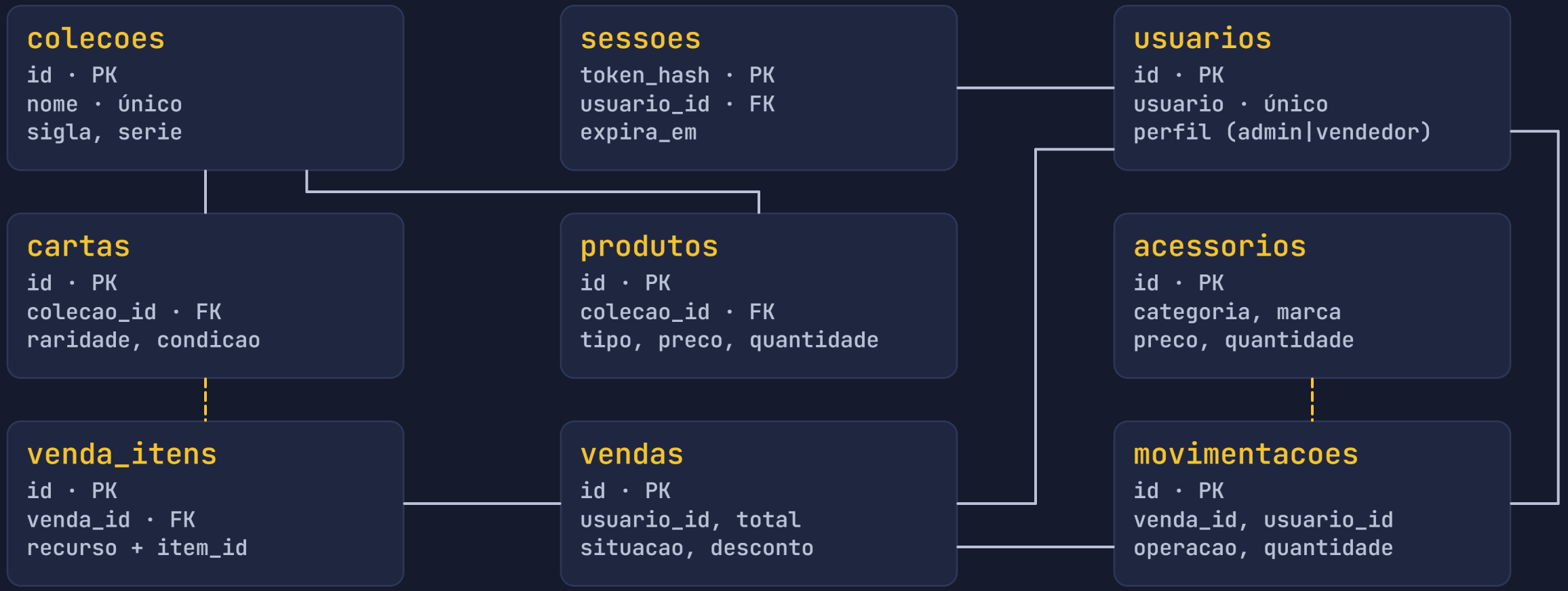
Código	Categoria	Requisito
RNF01	Segurança	Senhas com hash scrypt e salt; sessão em cookie HttpOnly por 7 dias
RNF02	Segurança	Login bloqueado por 1 minuto após 5 tentativas erradas
RNF03	Segurança	Perfil verificado em toda rota da API, não só escondendo botões
RNF04	Integridade	Venda, itens, saldo e movimentações gravados na mesma transação
RNF05	Portabilidade	PostgreSQL ou SQLite sem mudar o código, pela variável DATABASE_URL
RNF06	Desempenho	Listas paginadas, índices por data e fotos reduzidas em WEBP
RNF07	Usabilidade	Interface em português, responsiva até telas de celular
RNF10	Rastreabilidade	Histórico legível mesmo após excluir o item ou o usuário
RNF13	Qualidade	25 testes no GitHub Actions a cada push, no SQLite e no PostgreSQL

Diagrama de casos de uso



DER · nove tabelas

Linha contínua: chave estrangeira, 1:N
Tracejada: ligação lógica (recurso + item_id)



Atividade · registrar venda

Amarelo: decisão
Azul: dentro da mesma transação

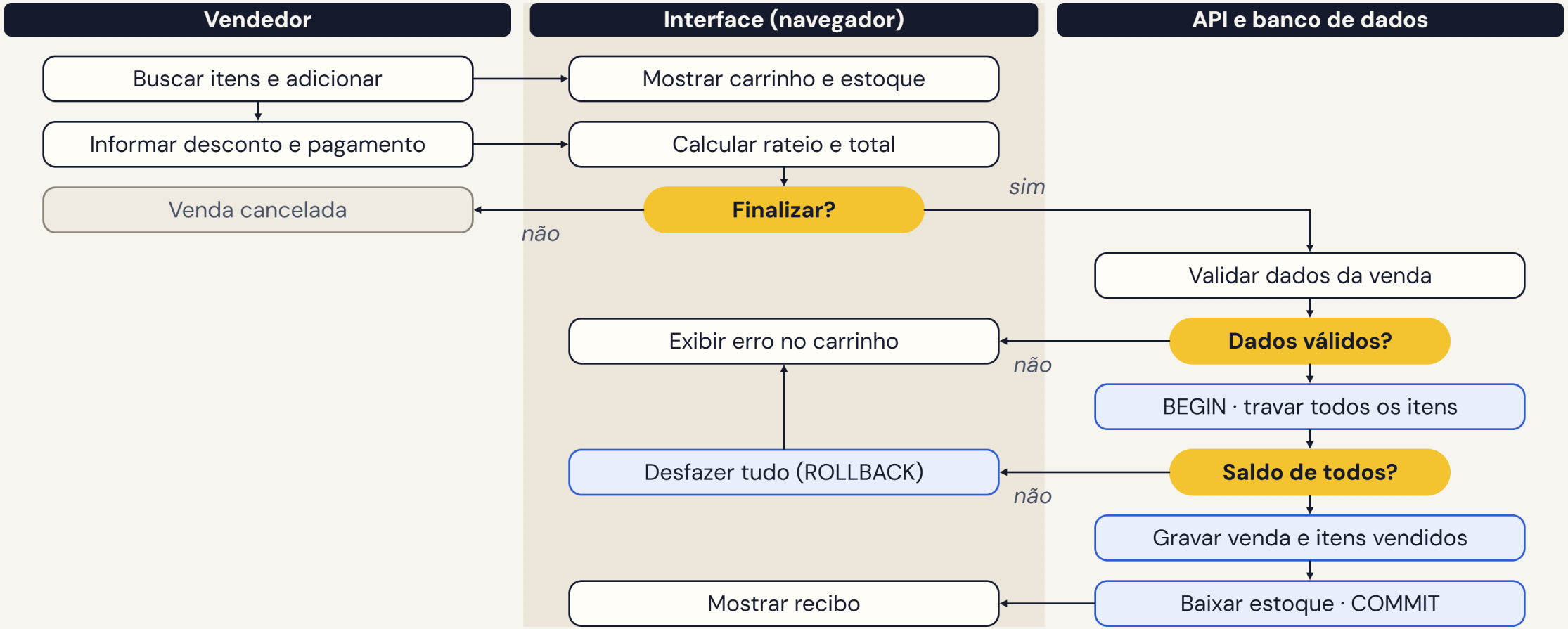
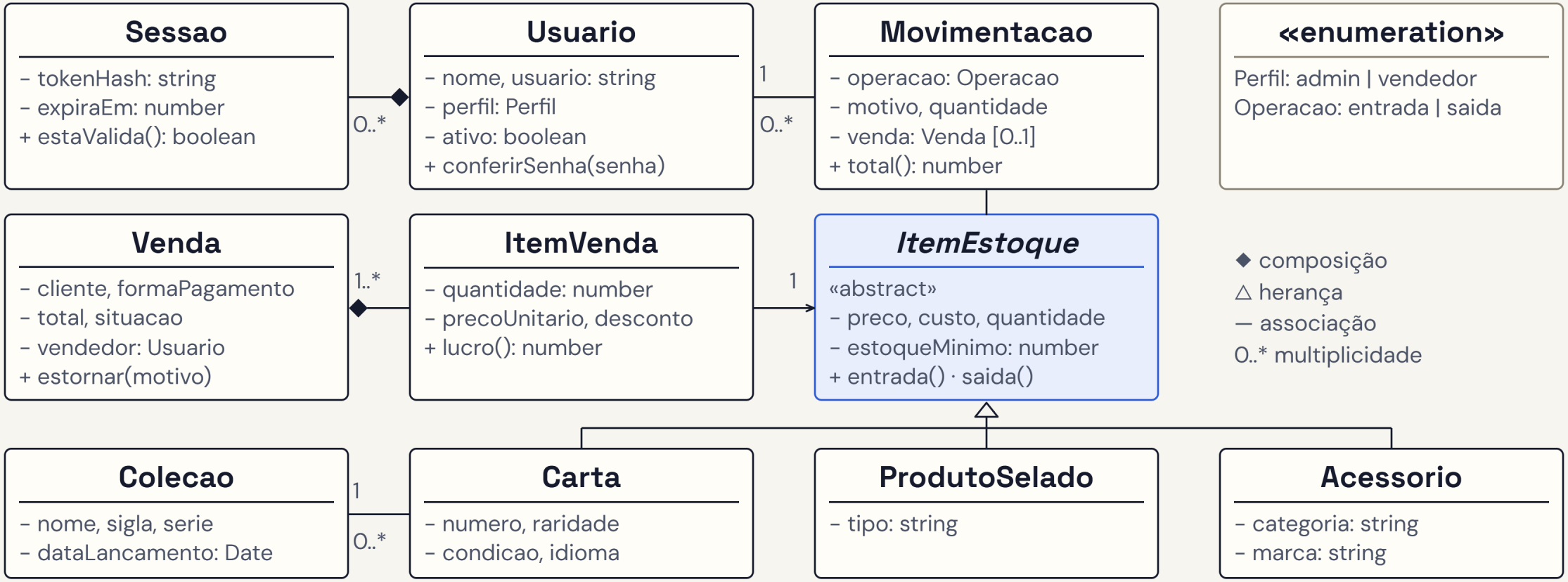
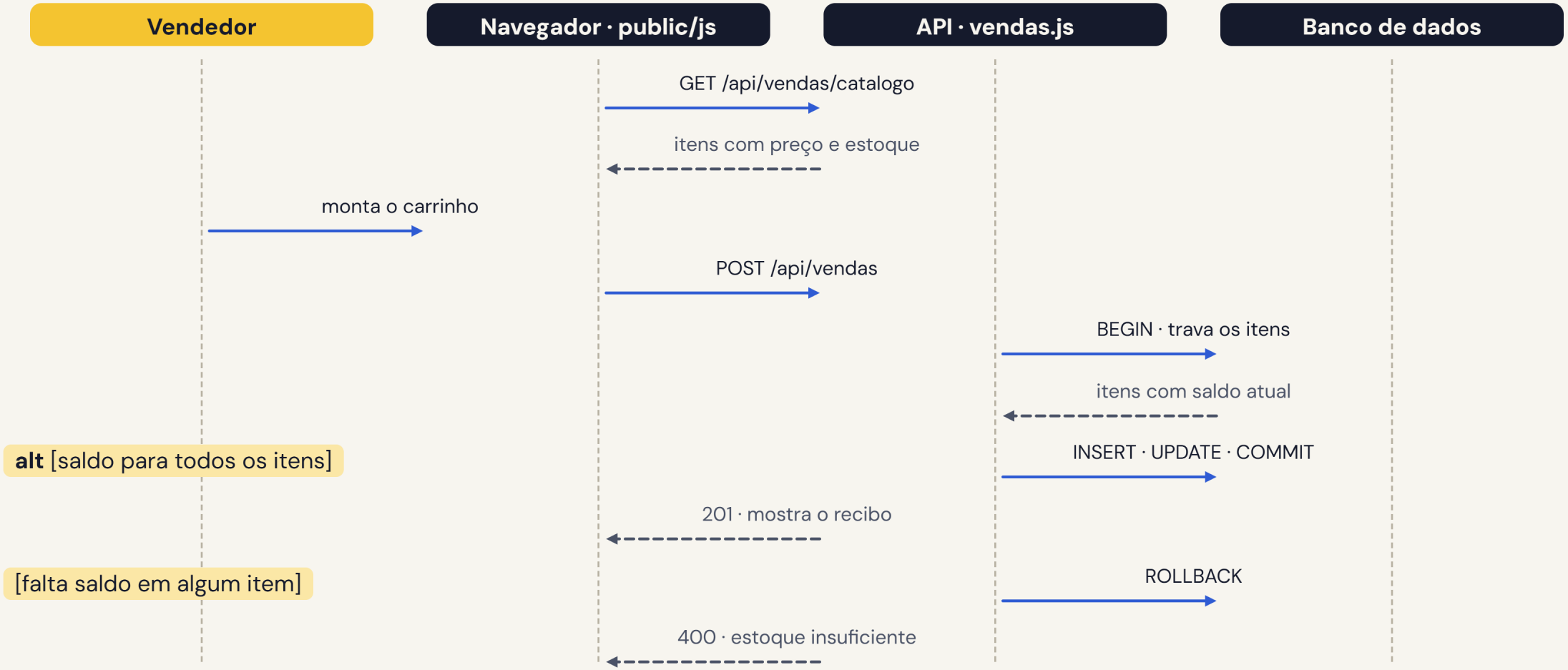


Diagrama de classes · entidades

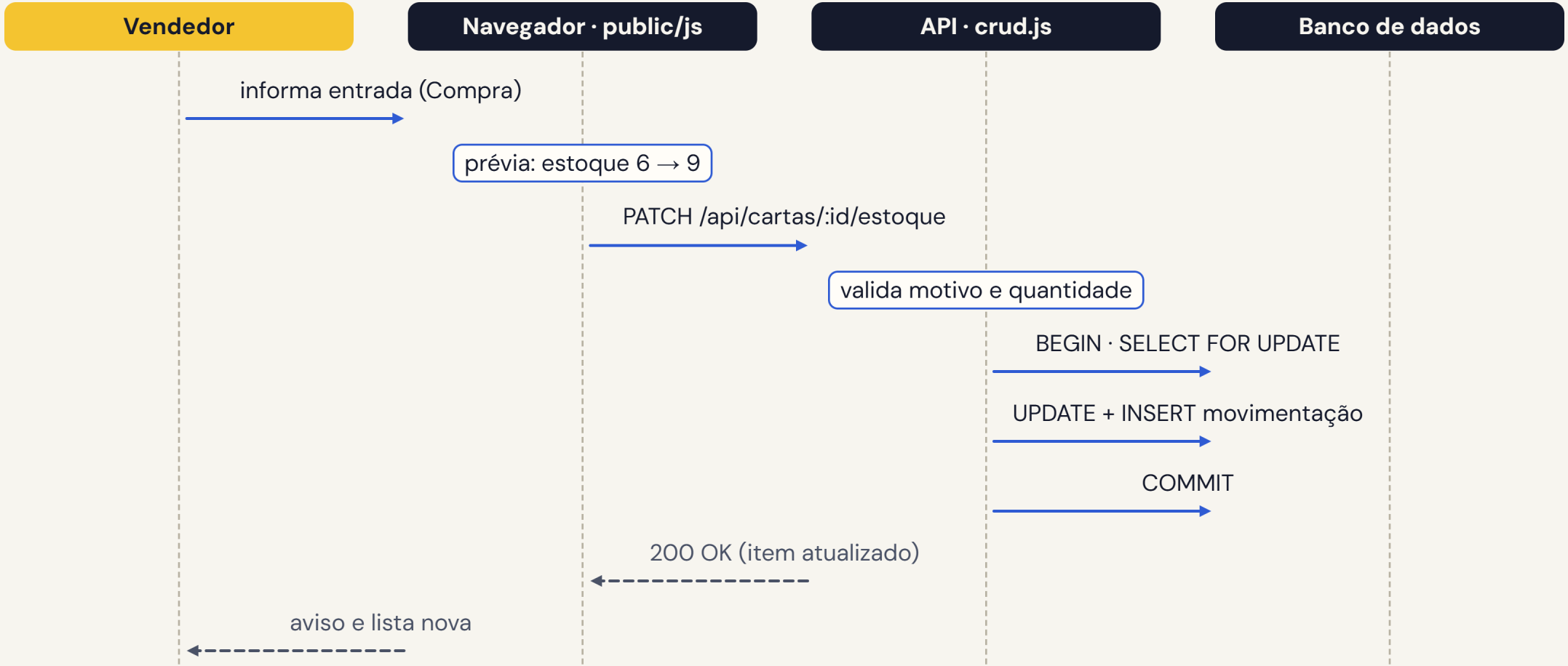


Sequência UC01 · Registrar venda

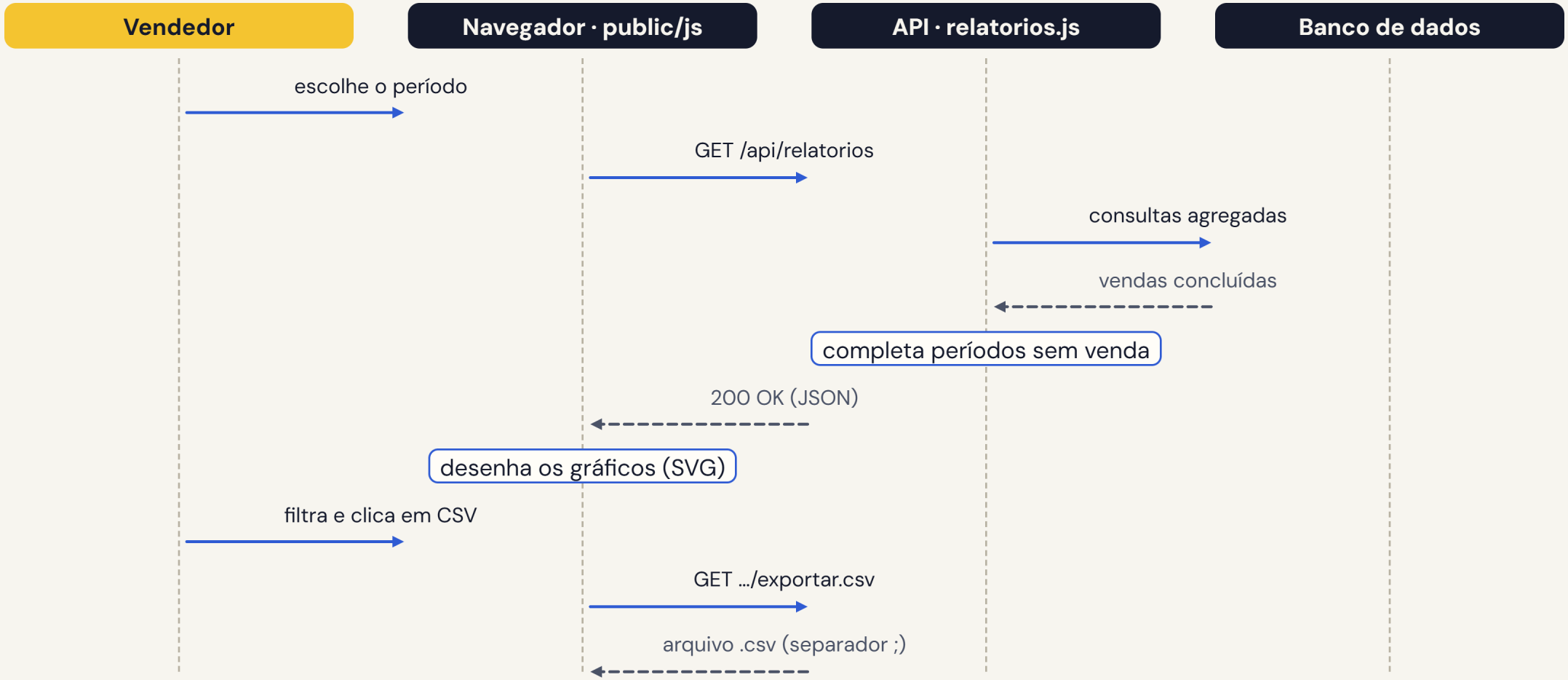


Hype Cards · Sequência · seta cheia: chamada · tracejada: resposta

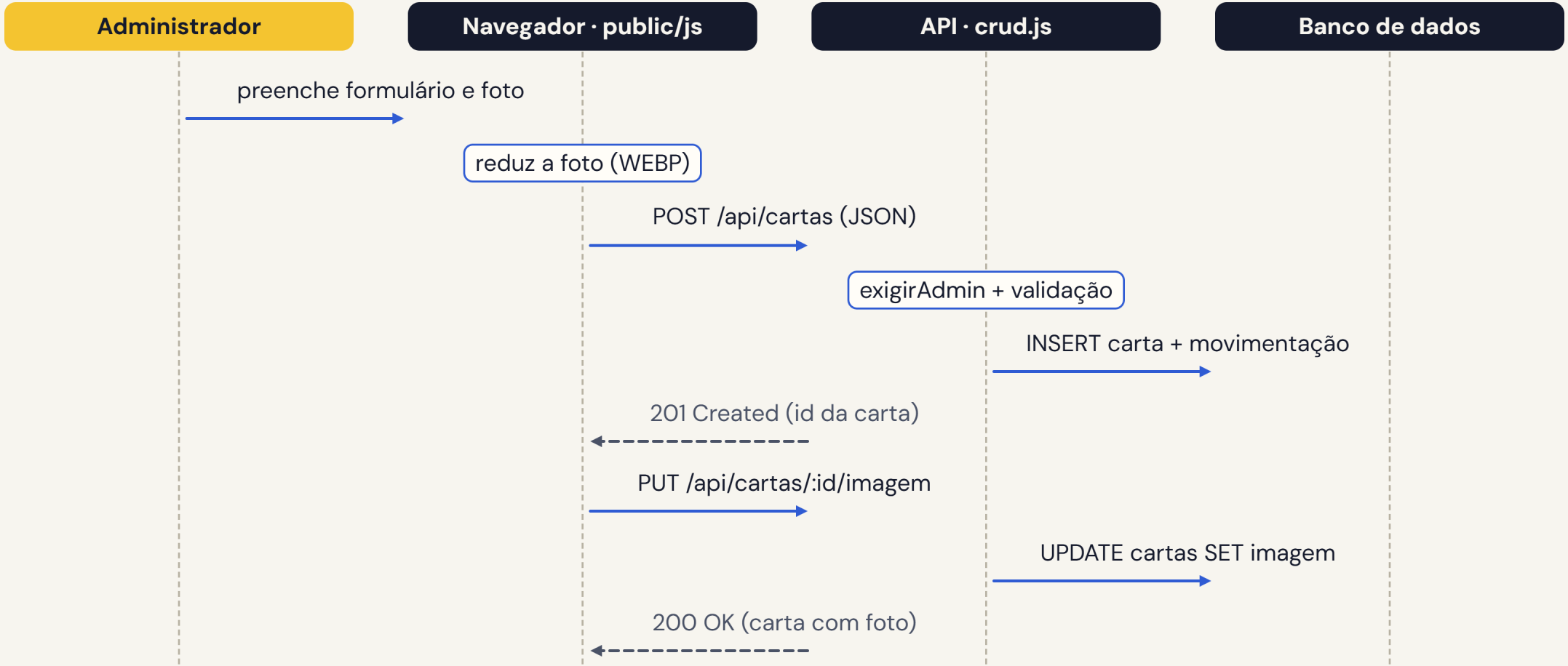
Sequência UC02 · Movimentação



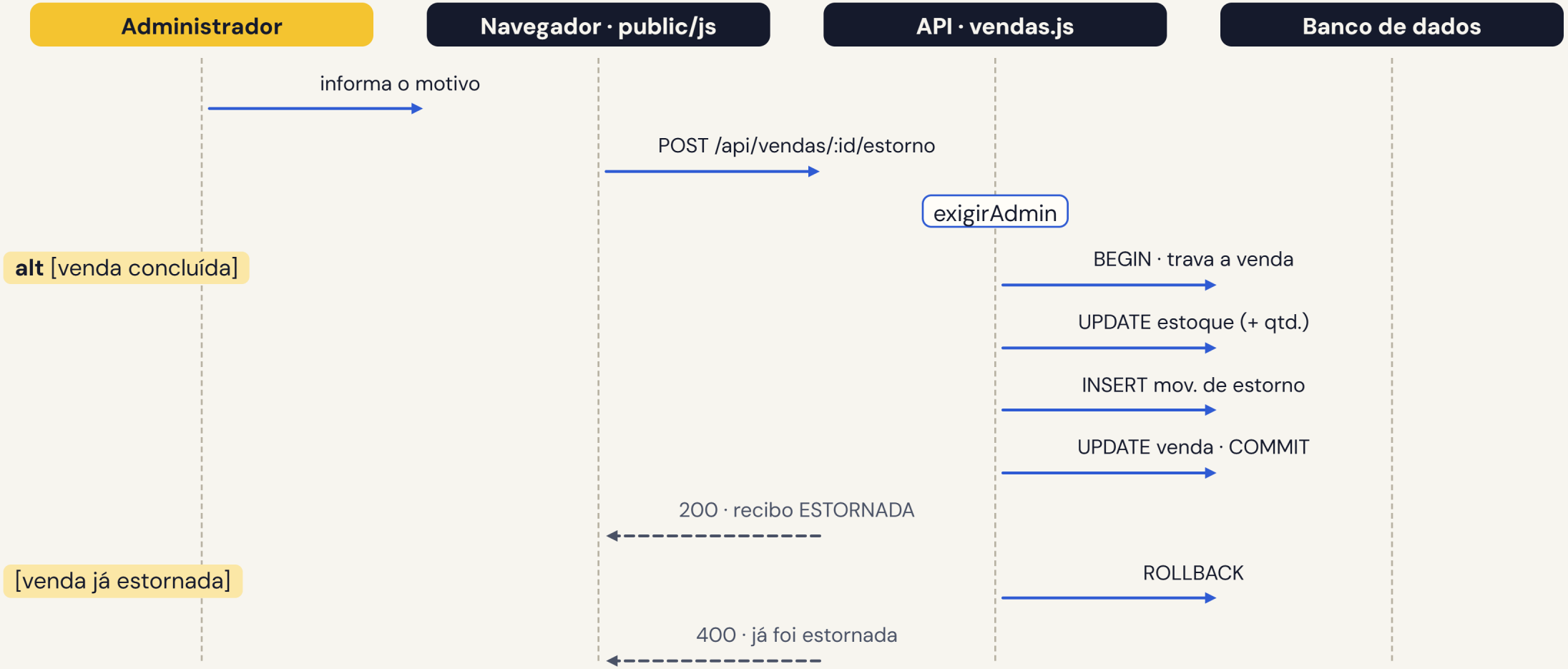
Sequência UC03 · Relatórios



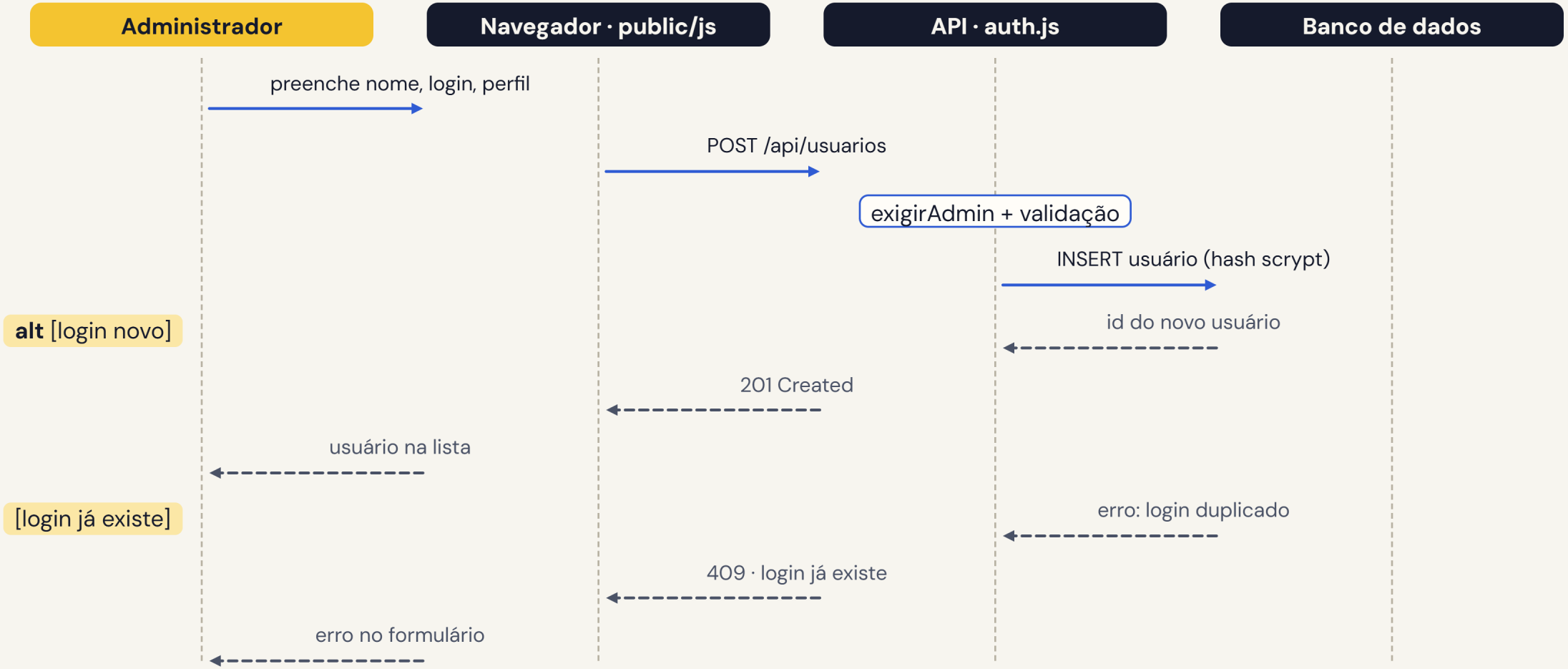
Sequência UC04 · Manter itens



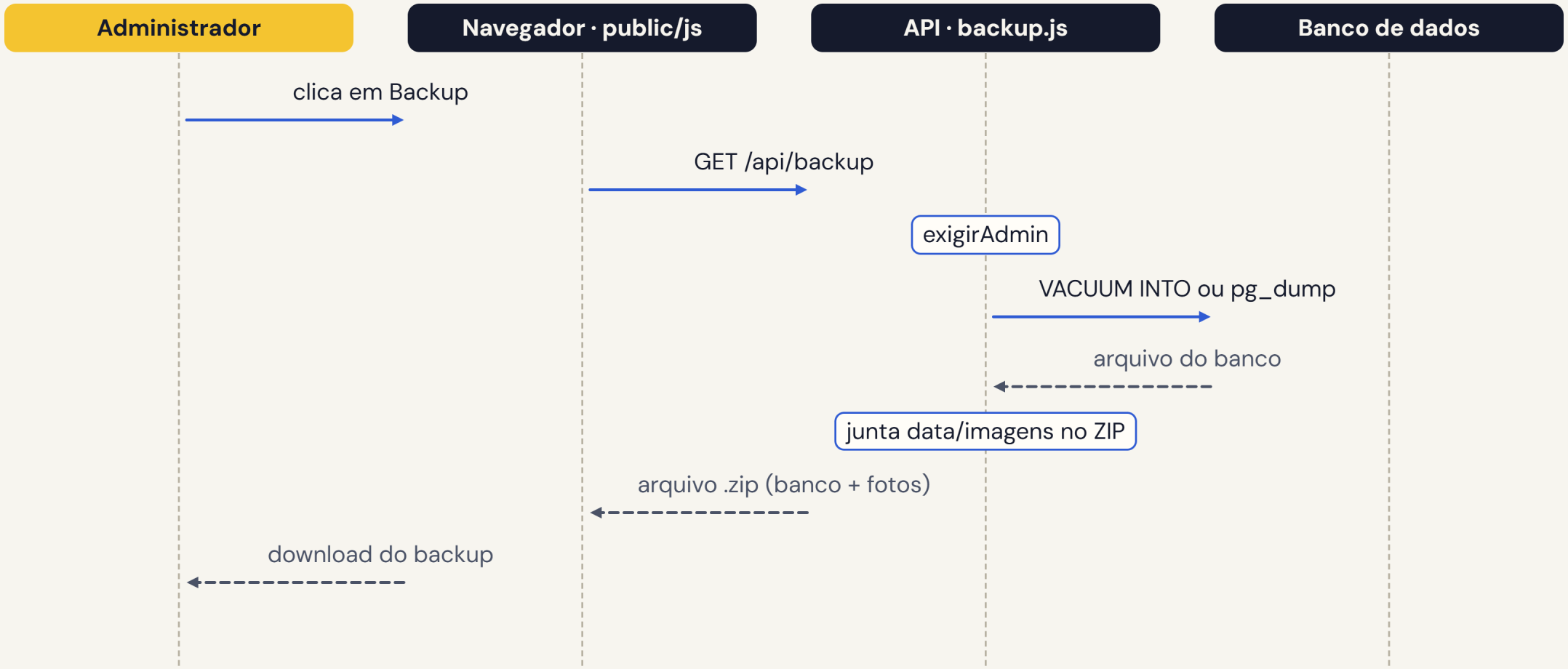
Sequência UC05 · Estornar venda



Sequência UC06 · Gerenciar usuários



Sequência UC07 · Realizar backup



O que o projeto ensinou

Modelar antes de codar

Campos de estoque comuns geraram um único módulo de cadastro para os três tipos.

Transações e concorrência

Venda com vários itens é tudo ou nada; travar sempre na mesma ordem evita impasse.

Dinheiro exige cuidado

Ratear desconto gera centavos de diferença; o resto vai para o último item.

Segurança no servidor

Senha e token nunca em texto puro; esconder um botão na tela não protege nada.

Bancos diferentes, SQL diferente

? contra $1, LIKE contra ILIKE: isolar o acesso a dados limitou a troca a poucos arquivos.

Testes e CI valem a pena

Os mesmos testes nos dois bancos acharam erros que só apareciam no PostgreSQL.

O sistema em números

24
requisitos funcionais

13
requisitos não funcionais

7
casos de uso, 2 atores

9
tabelas no banco de dados

Modelagem completa em UML: casos de uso, DER, atividade, classes e um diagrama de sequência por caso de uso.

Segurança e rastreabilidade

Senhas com scrypt

Hash com salt; nunca em texto puro. Troca obrigatória no primeiro acesso.

Sessão em cookie

HttpOnly e SameSite=Lax, 7 dias; no banco, só o hash SHA-256 do token.

Contra força bruta

5 tentativas erradas do mesmo usuário e IP bloqueiam o login por 1 minuto.

Perfil no servidor

Toda rota da API confere o perfil; vendedor em rota de admin recebe 403.

Usuários protegidos

Não dá para excluir a si mesmo nem remover o último administrador ativo.

Estorno sem apagar

A venda fica “estornada”, com motivo, data e responsável; os itens voltam ao estoque.

Objetivos atingidos

Estoque das três categorias, vendas com carrinho, recibo e estorno, histórico auditável e relatórios com gráficos, rodando em PostgreSQL ou SQLite.

Obrigado!